# Imports

In [1]:
```
import os
import zipfile
import pickle
import warnings
from pathlib import Path
from datetime import datetime
import numpy as np                      # numpy>=1.26
import pandas as pd                     # pandas>=2.0
import matplotlib.pyplot as plt         # matplotlib>=3.8
import seaborn as sns                   # seaborn>=0.13
#%pip install pyarrow
import pyarrow


import lightgbm as lgb                  # lightgbm>=4.0
from sklearn.metrics import (
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_auc_score,
)                                       # scikit-learn>=1.4

warnings.filterwarnings("ignore")
print(f"numpy={np.__version__}  pandas={pd.__version__}  lightgbm={lgb
```

```
numpy=2.2.6  pandas=2.3.3  lightgbm=4.6.0
```

In [2]:
```
import os, subprocess

REPO_URL = "https://github.com/tongyuguo/HelpHerInvest.git"
REPO_DIR = Path("HelpHerInvest")
data_dir = REPO_DIR / "Data"

def clone_or_pull():
    if (REPO_DIR / ".git").is_dir():
        subprocess.run(["git", "-C", str(REPO_DIR), "pull"])
    else:
        subprocess.run(["git", "clone", REPO_URL])

clone_or_pull()
```

```
Already up to date.
```

# Configuration

In [3]:
```
# PATHS
```

```
DATA_ZIP_PATH   = data_dir / "final_dataset_20260224v2.csv.zip"
DATA_CSV_NAME   = "final_dataset_20260224v2.csv"

ARTIFACTS_DIR   = REPO_DIR / "artifacts"
MODEL_PATH      = ARTIFACTS_DIR / "lgbm_model.pkl"
TRAIN_DATA_PATH = ARTIFACTS_DIR / "train_ranked.parquet"
VAL_DATA_PATH   = ARTIFACTS_DIR / "val_ranked.parquet"
TEST_DATA_PATH  = ARTIFACTS_DIR / "test_ranked.parquet"
ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)

# SPLIT FRACTIONS
TRAIN_FRAC = 0.70
VAL_FRAC   = 0.15
TEST_FRAC  = 0.15

# TARGET COLUMN
TARGET_COL = "y"           # 1 if fwd_excess > 0
DATE_COL   = "Date"
TICKER_COL = "Ticker"

# COLUMNS TO EXCLUDE FROM FEATURES
NON_FEATURE_COLS = [DATE_COL, TICKER_COL, "fwd_excess", "fwd_return",

# FEATURE COLUMNS
RANK_COLS = [
    "mom_1m", "mom_3m", "mom_6m", "mom_12m", "mom_12m_ex_1m",
    "rel_3m_spy", "rel_6m_spy", "rel_12m_spy",
    "vol_3m", "vol_6m",
    "drawdown_6m", "drawdown_12m",
    "pct_above_200dma",
]

#  LIGHTGBM HYPERPARAMETERS
LGBM_PARAMS = dict(
    n_estimators  = 1000,
    learning_rate = 0.01,
    max_depth     = 12,
    num_leaves    = 31,
    class_weight  = "balanced",
    random_state  = 42,
)
```

## Data Loading

In [4]:
```
def load_data(zip_path: Path, csv_name: str) -> pd.DataFrame:

    with zipfile.ZipFile(zip_path) as z:
        with z.open(csv_name) as f:
            df = pd.read_csv(f)

    df[DATE_COL] = pd.to_datetime(df[DATE_COL])
```

```
    tickers = df[TICKER_COL].unique()
    print(f"Loaded  : {df.shape[0]} rows  x  {df.shape[1]} cols")
    print(f"Tickers : {len(tickers)}")
    print(f"Date range: {df[DATE_COL].min().date()}  →  {df[DATE_COL].
    return df


def save_data(df: pd.DataFrame, path: Path, fmt: str = "parquet") -> N
    path.parent.mkdir(parents=True, exist_ok=True)
    if fmt == "parquet":
        df.to_parquet(path, index=False)
    elif fmt == "csv":
        df.to_csv(path, index=False)
    elif fmt == "pickle":
        df.to_pickle(path)
    else:
        raise ValueError(f"Unsupported fmt={fmt!r}. Choose parquet | c
    print(f"Saved  {df.shape} → {path}")


def load_saved_data(path: Path) -> pd.DataFrame:
    ext = path.suffix.lower()
    if ext == ".parquet":
        df = pd.read_parquet(path)
    elif ext == ".csv":
        df = pd.read_csv(path)
    elif ext in (".pkl", ".pickle"):
        df = pd.read_pickle(path)
    else:
        raise ValueError(f"Unknown extension {ext}. Supported: .parque
    print(f"Loaded {df.shape} ← {path}")
    return df


df_raw = load_data(DATA_ZIP_PATH, DATA_CSV_NAME)
```

```
Loaded  : 302024 rows  x  18 cols
Tickers : 1993
Date range: 2010-02-28  →  2026-02-28
```

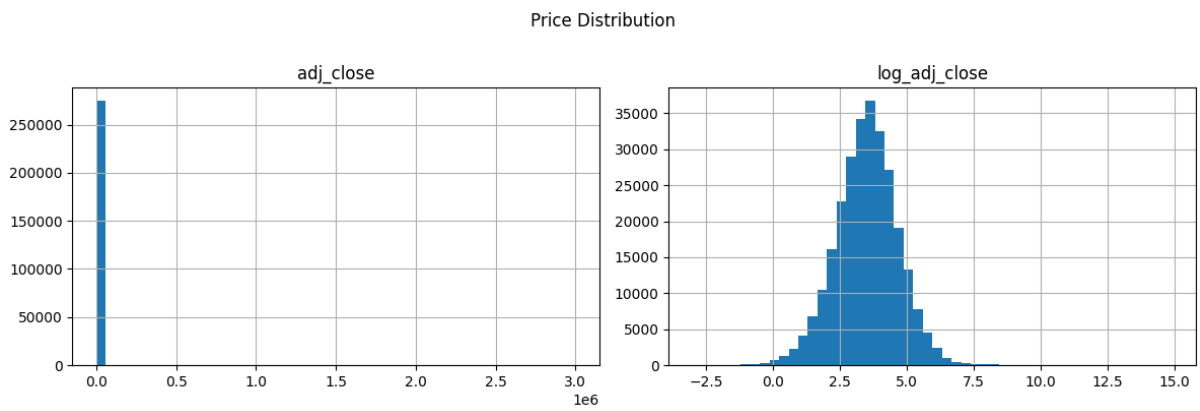
# Preprocessing

In [5]:

```
def preprocess_data(df: pd.DataFrame) -> pd.DataFrame:
    n_before = len(df)
    df_clean = df.dropna().copy()
    n_after  = len(df_clean)
    print(f"Dropped {n_before - n_after:,} rows with NaNs  ({n_before:

    # Binary target
    df_clean["y"] = (df_clean["fwd_excess"] > 0).astype(int)
    pos = df_clean["y"].sum()
```

```
    print(f"Class balance  →  positive: {pos}  ({pos/n_after:.1%})  |

    # Log price feature
    df_clean["log_adj_close"] = np.log(df_clean["adj_close"])

    return df_clean


df_clean = preprocess_data(df_raw)

# Adjusted close price and log price distributions
df_clean[["adj_close", "log_adj_close"]].hist(figsize=(12, 4), bins=50
plt.suptitle("Price Distribution", y=1.01)
plt.tight_layout()
plt.show()
```

```
Dropped 27,513 rows with NaNs  (302,024 → 274,511)
Class balance  →  positive: 132974  (48.4%)  |  negative: 141537  (51.
6%)
```




# Feature Engineering — Cross-Sectional Ranking

In [6]:
```
def apply_cross_sectional_rank(df: pd.DataFrame, rank_cols: list, date
    df_rank = df.copy()
    for col in rank_cols:
        if col not in df_rank.columns:
            print(f"  Warning: column {col!r} not found — skipping.")
            continue
        df_rank[col] = df_rank.groupby(date_col)[col].rank(pct=True)

    # Quintile label for the forward return (0 = worst, 4 = best)
    df_rank["fwd_rank"] = df_rank.groupby(date_col)["fwd_excess"].rank
    df_rank["target"]   = pd.cut(
        df_rank["fwd_rank"],
        bins=[0, .2, .4, .6, .8, 1.0],
        labels=[0, 1, 2, 3, 4],
    )
    return df_rank
```

```
def retrieve_features(df: pd.DataFrame, exclude_cols: list = NON_FEATU
    feature_cols = sorted([c for c in df.columns if c not in exclude_c
    print(f"Feature count: {len(feature_cols)}")
    return feature_cols



df_rank = apply_cross_sectional_rank(df_clean, RANK_COLS)
FEATURE_COLS = retrieve_features(df_rank)
print("Sample features:", FEATURE_COLS[:10])
```

```
Feature count: 15
Sample features: ['adj_close', 'drawdown_12m', 'drawdown_6m', 'log_adj_
close', 'mom_12m', 'mom_12m_ex_1m', 'mom_1m', 'mom_3m', 'mom_6m', 'pct_
above_200dma']
```

## Train / Val / Test Split

In [7]:

```
def time_split(
    df: pd.DataFrame,
    date_col: str  = DATE_COL,
    train_frac: float = TRAIN_FRAC,
    val_frac:   float = VAL_FRAC,
    test_frac:  float = TEST_FRAC,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    if abs(train_frac + val_frac + test_frac - 1.0) > 1e-8:
        raise ValueError("Fractions must sum to 1.0")

    unique_dates = df[date_col].drop_duplicates().sort_values().reset_
    n = len(unique_dates)
    i_train = int(n * train_frac)
    i_val   = int(n * (train_frac + val_frac))

    train_df = df[df[date_col].isin(unique_dates.iloc[:i_train])].copy
    val_df   = df[df[date_col].isin(unique_dates.iloc[i_train:i_val])]
    test_df  = df[df[date_col].isin(unique_dates.iloc[i_val:])].copy()

    for name, split in [("Train", train_df), ("Val", val_df), ("Test",
        print(f"{name:5s}: {len(split):>7,} rows  |  "
              f"{split[date_col].min().date()}  →  {split[date_col].ma
    return train_df, val_df, test_df



train_df, val_df, test_df = time_split(df_rank)

X_train, y_train = train_df[FEATURE_COLS], train_df[TARGET_COL]
X_val,   y_val   = val_df[FEATURE_COLS],   val_df[TARGET_COL]
X_test,  y_test  = test_df[FEATURE_COLS],  test_df[TARGET_COL]

# Save splits for reproducibility and future inference
```

```
#save_data(train_df, TRAIN_DATA_PATH)
#save_data(val_df,   VAL_DATA_PATH)
#save_data(test_df,  TEST_DATA_PATH)
```

```
Train: 176,701 rows  |  2011-01-31  →  2021-05-31
Val  :  47,488 rows  |  2021-06-30  →  2023-08-31
Test :  50,322 rows  |  2023-09-30  →  2025-11-30
```

# Model Training

In [8]:

```
def train_lgbm(
    X_train: pd.DataFrame,
    y_train: pd.Series,
    X_val:   pd.DataFrame,
    y_val:   pd.Series,
    params:  dict = LGBM_PARAMS,
) -> lgb.LGBMClassifier:
    model = lgb.LGBMClassifier(**params)
    model.fit(
        X_train, y_train,
        eval_set=[(X_val, y_val)],
    )
    print(f"Training complete — best iteration: {model.best_iteration_
    return model

lgb_model = train_lgbm(X_train, y_train, X_val, y_val)
```

```
[LightGBM] [Info] Number of positive: 86755, number of negative: 89946
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.029116 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 176701, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=
0.000000
[LightGBM] [Info] Start training from score 0.000000
Training complete — best iteration: 0
```

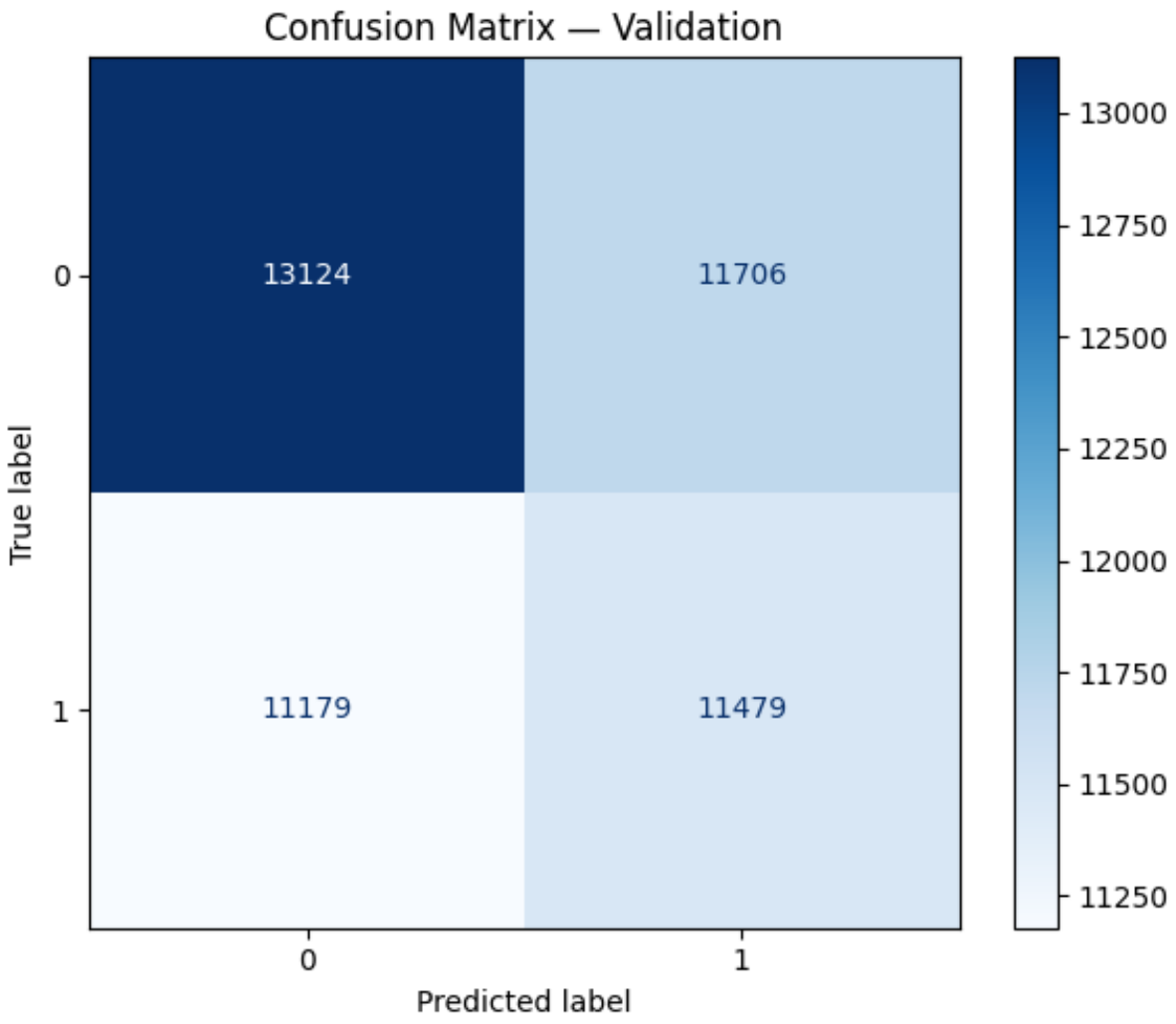
# Evaluation

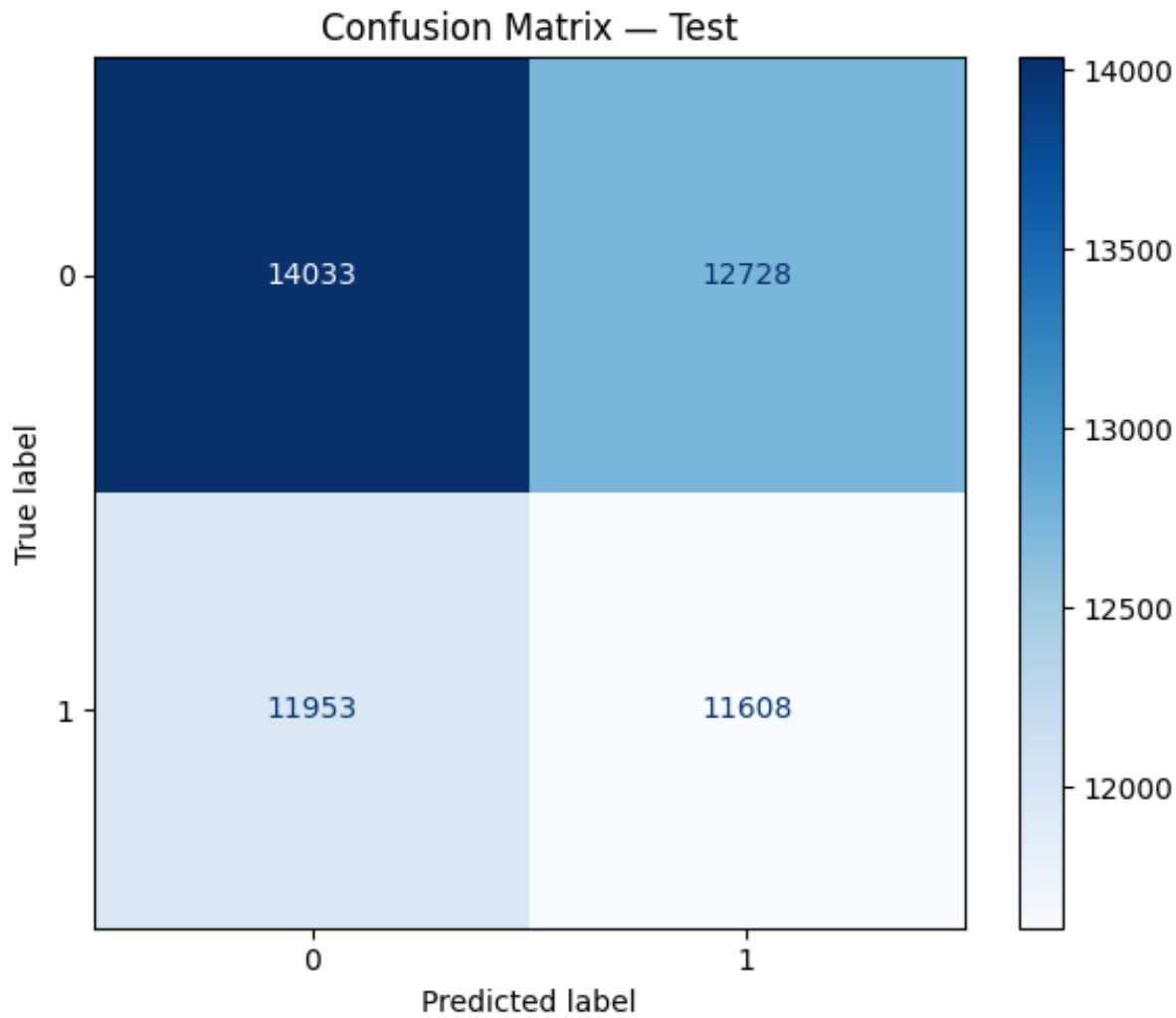
In [9]:

```
def evaluate(
    model,
    X: pd.DataFrame,
    y: pd.Series,
    split_name: str = "Set",
) -> dict:
    y_pred = model.predict(X)
    probs  = model.predict_proba(X)[:, 1]
    f1     = round(f1_score(y, y_pred, average="macro"), 4)
```

```
    auc    = round(roc_auc_score(y, probs, average="macro"), 4)

    print(f"── {split_name} ──")
    print(classification_report(y, y_pred))
    print(f"Macro F1:  {f1}")
    print(f"ROC-AUC:   {auc}")
    return {"f1_macro": f1, "roc_auc": auc}


def plot_confusion_matrix(
    model,
    X: pd.DataFrame,
    y: pd.Series,
    title: str = "Confusion Matrix",
) -> pd.DataFrame:
    y_pred = model.predict(X)
    cm     = confusion_matrix(y, y_pred)
    cm_df  = pd.DataFrame(cm, index=["Actual 0", "Actual 1"], columns=

    fig, ax = plt.subplots(figsize=(6, 5))
    ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
        ax=ax, cmap="Blues", values_format="d"
    )
    ax.set_title(title)
    plt.tight_layout()
    plt.show()
    return cm_df


val_metrics  = evaluate(lgb_model, X_val,  y_val,  "Validation Set")
test_metrics = evaluate(lgb_model, X_test, y_test, "Test Set")

plot_confusion_matrix(lgb_model, X_val,  y_val,  "Confusion Matrix — V
plot_confusion_matrix(lgb_model, X_test, y_test, "Confusion Matrix — T
```

```
── Validation Set ──
              precision    recall  f1-score   support

           0       0.54      0.53      0.53     24830
           1       0.50      0.51      0.50     22658

    accuracy                           0.52     47488
   macro avg       0.52      0.52      0.52     47488
weighted avg       0.52      0.52      0.52     47488

Macro F1:  0.5175
ROC-AUC:   0.5229
── Test Set ──
              precision    recall  f1-score   support

           0       0.54      0.52      0.53     26761
           1       0.48      0.49      0.48     23561

    accuracy                           0.51     50322
   macro avg       0.51      0.51      0.51     50322
weighted avg       0.51      0.51      0.51     50322

Macro F1:  0.5084
ROC-AUC:   0.5133
```


Confusion Matrix — Validation
13124
11706
11179
11479
0
1
True label
0
1
Predicted label
13000
12750
12500
12250
12000
11750
11500
11250




Out[9]:

| | Pred 0 | Pred 1 |
|---|---|---|
| **Actual 0** | 14033 | 12728 |
| **Actual 1** | 11953 | 11608 |

## Save & Load Model

In [10]:

```
def save_model(model, path: Path) -> None:
    """
    Serialize the model to disk using pickle.
    """
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "wb") as f:
        pickle.dump(model, f, protocol=pickle.HIGHEST_PROTOCOL)
    size_kb = path.stat().st_size / 1024
    print(f"Model saved  → {path}  ({size_kb:.1f} KB)  |  {datetime.no


def load_model(path: Path):
    """
```

```
    Load the pickled model from disk.
    """
    with open(path, "rb") as f:
        model = pickle.load(f)
    print(f"Model loaded ← {path}")
    return model


# Save the model
save_model(lgb_model, MODEL_PATH)

# Reload and verify
lgb_model_loaded = load_model(MODEL_PATH)
print("Verification:")
evaluate(lgb_model_loaded, X_val, y_val, "Validation (reloaded model)"
```

```
Model saved  → HelpHerInvest/artifacts/lgbm_model.pkl  (3455.5 KB)  |
2026-04-24 23:30:27
Model loaded ← HelpHerInvest/artifacts/lgbm_model.pkl
Verification:
── Validation (reloaded model) ──
              precision    recall  f1-score   support

           0       0.54      0.53      0.53     24830
           1       0.50      0.51      0.50     22658

    accuracy                           0.52     47488
   macro avg       0.52      0.52      0.52     47488
weighted avg       0.52      0.52      0.52     47488

Macro F1:  0.5175
ROC-AUC:   0.5229
```

Out[10]: {'f1_macro': 0.5175, 'roc_auc': 0.5229}

# NLP Ticker Selection

In [11]:

```
# create unique ticker dataframe

def create_unique_ticker_df(df):
    if "Ticker" not in df.columns:
        raise ValueError("Column 'Ticker' not found in dataframe.")

    unique_ticker = (
        df["Ticker"]
        .dropna()
        .astype(str)
        .str.upper()
        .str.strip()
        .drop_duplicates()
        .sort_values()
```

```
        .reset_index(drop=True)
        .to_frame(name="Ticker")
    )

    return unique_ticker

unique_ticker = create_unique_ticker_df(df_rank)

print(unique_ticker.head())
print("Number of unique tickers:", len(unique_ticker))
```

```
  Ticker
0      A
1     AA
2    AAL
3   AAON
4    AAP
Number of unique tickers: 1913
```

In [12]:

```
from pprint import pprint

#API
from dotenv import load_dotenv
import os
load_dotenv()

from openai import OpenAI
import pandas as pd
import io
import json

client = OpenAI()

def get_30_unique_tickers(unique_ticker: pd.DataFrame, user_input: str

    if "Ticker" not in unique_ticker.columns:
        raise ValueError("unique_ticker must contain a 'Ticker' column

    # Clean ticker column
    valid_tickers = (
        unique_ticker["Ticker"]
        .dropna()
        .astype(str)
        .str.upper()
        .str.strip()
        .drop_duplicates()
    )

    valid_ticker_set = set(valid_tickers)

    # Convert dataframe to in-memory CSV
    ticker_buffer = io.BytesIO()
    valid_tickers.to_frame(name="Ticker").to_csv(ticker_buffer, index=
```

```
    ticker_buffer.seek(0)
    ticker_buffer.name = "unique_ticker.csv"

    # Upload in-memory file
    uploaded_file = client.files.create(file=ticker_buffer, purpose="u

    # Call model
    response = client.responses.create(
        model=model,
        input=[
            {
                "role": "developer",
                "content": [
                    {
                        "type": "input_text",
                        "text": (
                            "You are selecting stock tickers for a dow
                            "You must use the attached CSV file as the
                            "Based on the user's interest, return exac
                            "Rules:\n"
                            "- Only return tickers that exist in the a
                            "- Return exactly 30 unique tickers.\n"
                            "- Do not include explanations.\n"
                            "- Do not include markdown.\n"
                            '- Return valid JSON in this exact format:
                        )
                    }
                ]
            },
            {
                "role": "user",
                "content": [
                    {"type": "input_file", "file_id": uploaded_file.id
                    {"type": "input_text", "text": user_input}
                ]
            }
        ]
    )

    # Parse JSON output
    try:
        result = json.loads(response.output_text)
        tickers = result["tickers"]
    except Exception as e:
        raise ValueError(f"Model output was not valid JSON: {response.

    # Clean + validate output
    cleaned = []
    seen = set()

    for t in tickers:
        t = str(t).upper().strip()
```

```
        if t in valid_ticker_set and t not in seen:
            cleaned.append(t)
            seen.add(t)

    if len(cleaned) != 30:
        raise ValueError(
            f"Expected 30 valid unique tickers, but got {len(cleaned)}
            f"Raw model output: {response.output_text}"
        )

    # Return dataframe
    selected_tickers_df = pd.DataFrame({"Ticker": cleaned})
    return selected_tickers_df
```

In [13]:

```
user_input = "I like the food industry."

selected_tickers_df = get_30_unique_tickers(unique_ticker, user_input)

print(selected_tickers_df)
print(selected_tickers_df.shape)
```

```
   Ticker
0      KO
1    COKE
2     KDP
3     GIS
4     KHC
5     HSY
6     ADM
7     CPB
8     CMG
9     DPZ
10    DRI
11   CAKE
12   COST
13     KR
14    BUD
15    HRL
16   BF-A
17    CAG
18   COLD
19   CAVA
20    EAT
21    GPK
22     CF
23   AGCO
24     BJ
25   ABEV
26   BROS
27   CHWY
28   CHRW
29   CPNG
(30, 1)
```

# Rank Selected Tickers

In [14]:
```
def rank_selected_tickers(
    selected_tickers_df: pd.DataFrame,
    feature_df: pd.DataFrame,
    model,
    feature_cols: list,
    ticker_col: str = "Ticker",
    date_col: str = "Date",
    as_of_date=None,
) -> pd.DataFrame:
    if ticker_col not in selected_tickers_df.columns:
        raise ValueError(f"selected_tickers_df must contain '{ticker_c

    missing_features = [c for c in feature_cols if c not in feature_df
    if missing_features:
        raise ValueError(f"feature_df is missing required feature colu

    if ticker_col not in feature_df.columns or date_col not in feature
        raise ValueError(f"feature_df must contain '{ticker_col}' and

    # Clean tickers in both dfs
    selected = selected_tickers_df.copy()
    selected[ticker_col] = (
        selected[ticker_col].astype(str).str.upper().str.strip()
    )

    df = feature_df.copy()
    df[ticker_col] = df[ticker_col].astype(str).str.upper().str.strip(
    df[date_col] = pd.to_datetime(df[date_col])

    # Determine scoring cutoff date
    if as_of_date is None:
        as_of_date = df[date_col].max()
    else:
        as_of_date = pd.to_datetime(as_of_date)

    # Keep only rows up to the as-of date
    df = df[df[date_col] <= as_of_date].copy()

    # Keep only selected tickers
    selected_set = set(selected[ticker_col])
    df = df[df[ticker_col].isin(selected_set)].copy()

    if df.empty:
        raise ValueError("No matching ticker rows found in feature_df

    # For each ticker, use the latest available row up to as_of_date
    df = df.sort_values([ticker_col, date_col])
    latest_rows = df.groupby(ticker_col, as_index=False).tail(1).copy(
```

```
    # Drop rows with missing model features
    before = len(latest_rows)
    latest_rows = latest_rows.dropna(subset=feature_cols).copy()
    dropped = before - len(latest_rows)
    if dropped > 0:
        print(f"Dropped {dropped} selected tickers due to missing feat

    if latest_rows.empty:
        raise ValueError("No selected tickers remain after dropping mi

    # Score with model
    X_score = latest_rows[feature_cols]
    latest_rows["score"] = model.predict_proba(X_score)[:, 1]

    # Rank descending
    ranked = latest_rows.sort_values("score", ascending=False).reset_i
    ranked["rank"] = ranked["score"].rank(method="first", ascending=Fa

    # Select output columns
    output_cols = [ticker_col, date_col, "score", "rank"]
    return ranked[output_cols]
```

In [15]:

```
ranked_tickers_df = rank_selected_tickers(
    selected_tickers_df=selected_tickers_df,
    feature_df=df_rank,
    model=lgb_model,
    feature_cols=FEATURE_COLS,
    ticker_col="Ticker",
    date_col="Date",
    as_of_date=None,   # uses latest available date
)

print(ranked_tickers_df.head(30))
```

| | Ticker | Date | score | rank |
|---|---|---|---|---|
| 0 | ABEV | 2025-11-30 | 0.527055 | 1 |
| 1 | HRL | 2025-11-30 | 0.521555 | 2 |
| 2 | BJ | 2025-11-30 | 0.515662 | 3 |
| 3 | KDP | 2025-11-30 | 0.511879 | 4 |
| 4 | DRI | 2025-11-30 | 0.510748 | 5 |
| 5 | COLD | 2025-11-30 | 0.509198 | 6 |
| 6 | AGCO | 2025-11-30 | 0.508622 | 7 |
| 7 | CAKE | 2025-11-30 | 0.503568 | 8 |
| 8 | ADM | 2025-11-30 | 0.503308 | 9 |
| 9 | BUD | 2025-11-30 | 0.502872 | 10 |
| 10 | CPNG | 2025-11-30 | 0.502186 | 11 |
| 11 | KR | 2025-11-30 | 0.500835 | 12 |
| 12 | CPB | 2025-11-30 | 0.500241 | 13 |
| 13 | DPZ | 2025-11-30 | 0.498206 | 14 |
| 14 | CF | 2025-11-30 | 0.497018 | 15 |
| 15 | HSY | 2025-11-30 | 0.493437 | 16 |
| 16 | CHRW | 2025-11-30 | 0.491018 | 17 |
| 17 | BROS | 2025-11-30 | 0.490934 | 18 |
| 18 | CHWY | 2025-11-30 | 0.488240 | 19 |
| 19 | BF-A | 2025-11-30 | 0.487387 | 20 |
| 20 | COKE | 2025-11-30 | 0.486214 | 21 |
| 21 | GPK | 2025-11-30 | 0.477477 | 22 |
| 22 | KHC | 2025-11-30 | 0.476631 | 23 |
| 23 | CMG | 2025-11-30 | 0.475766 | 24 |
| 24 | CAG | 2025-11-30 | 0.475213 | 25 |
| 25 | CAVA | 2025-11-30 | 0.471600 | 26 |
| 26 | GIS | 2025-11-30 | 0.465214 | 27 |
| 27 | EAT | 2025-11-30 | 0.453657 | 28 |
| 28 | KO | 2025-11-30 | 0.450116 | 29 |
| 29 | COST | 2025-11-30 | 0.422124 | 30 |